

Crossover Helps Only Where the Search Space Has Building Blocks: A Trap-Function Control and a Real NAS Negative

Vasili Gavrilov*

2026

Abstract

Evolutionary neural architecture search recombines architectures with crossover, but random search is a strong baseline and it is unclear whether crossover adds anything of its own. We test one rule: crossover helps when the search space is built from separable blocks that recombination can assemble. As a positive control, on deceptive trap functions (independent blocks by design) crossover beats mutation, and its advantage grows with the number of blocks, from 1.8 times at four blocks to sixteen blocks, where mutation reaches the optimum 28 times in 200 runs and crossover 199. On NAS-Bench-101 (423,624 tabulated cells, so fitness is a table lookup and thousands of paired runs are cheap), no method meaningfully beats random: random and all four crossovers land within a few tenths of a percent of each other, inside the benchmark’s own training noise. Thousands of paired seeds still resolve a consistent ordering, and it does not favor structure. Every crossover that recombines the cell’s structure (wiring versus labeling, node roles, paths) is consistently worse than a plain structure-blind uniform crossover, and two fall slightly below random at large budget. So the cell space has no building-block structure to exploit; the small, consistent edge of uniform crossover over random comes from diversity, not structure, which we confirm by measuring population diversity and by a population-size sweep. The result is the familiar one, that random search is hard to beat, now with a mechanism and a positive control that shows when recombination does help. Every number regenerates from one command.

1 Introduction

Neural architecture search (NAS) automates a network’s shape; methods differ mainly in search strategy: reinforcement learning [5], gradient relaxation [6], Bayesian optimization, random search, and evolution [8]. Evolutionary NAS mutates and recombines architectures, training each candidate by back-propagation [7]. Two facts motivate us: plain random search is a strong baseline [4, 9], and the leading evolutionary method drops crossover entirely, keeping only mutation with aging [7].

Crossover [2] is meant to recombine *building blocks*, partial solutions that can be assembled across parents. So the real question is not whether crossover helps, but whether the space has building blocks to recombine. We answer with one rule and two testbeds. A positive control fixes what the rule means: deceptive trap functions, which are separable into blocks by construction, where crossover must and does win. Then the real test: NAS-Bench-101, whose cell has two natural dimensions, a wiring dimension and a labeling dimension, plus a path view. We ask whether a crossover that respects that structure beats one that ignores it. As a hypothesis, H_1

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including a genetic-algorithm optimizer built for industrial use in 2002. Reference implementation: <https://github.com/Anode1/SMBPANN>.

Table 1: Concatenated deceptive traps ($K = 4$ bits per block), mean generations to the global optimum and how many of 200 seeds reach it. Crossover’s advantage over mutation grows with the number of blocks M , the signature of a genuine building-block effect, until mutation essentially fails and crossover still solves it.

blocks M	mutation-only	crossover + mutation	speedup
4	9.2 gens, 200/200	5.1 gens, 200/200	$1.8\times$
8	70.8 gens, 200/200	14.2 gens, 200/200	$5\times$
12	257 gens, 185/200	22.6 gens, 200/200	$11\times$
16	363 gens, 28/200	42.3 gens, 199/200	mutation fails

says a crossover that recombines the cell’s building blocks (its wiring and labeling, its node roles, its paths) beats a structure-blind uniform crossover; the null H_0 says it does not, because the real cell space carries no such recombinable structure. The intuition behind H_1 is strong, and our trap control shows the effect it predicts is real when blocks exist. Yet on NAS-Bench-101 the data does not merely fail to reject H_0 : the structure-aware operators are consistently worse than the structure-blind one, and at large budget worse than random search. The recombination logic comes from a self-modifying back-propagation engine that grew out of the author’s 1997 seminar thesis [1]; on NAS-Bench-101 we move fitness onto the table so training noise cannot confound the comparison.

2 Positive control: crossover on separable building blocks

Before the real space, we fix what a building-block advantage looks like on a problem *designed* to have one. The genome is M concatenated deceptive trap blocks of $K = 4$ bits: a block scores K if all its bits are 1, else $K - 1$ minus its number of ones. That gradient lures a bit-flipping search toward all-zero, so the all-ones optimum is reachable only by assembling correctly-solved blocks, which is what one-point crossover recombines and a mutation-only search cannot climb to [3]. We run elitism with tournament selection and one-point crossover ($p = 0.9$) against the identical search with mutation only (population 1000, 200 seeds, cap 500 generations), and scan M .

Table 1 is the control: when the space is separable into building blocks, crossover is not marginally but decisively better, and the gap widens with the amount of structure until mutation alone cannot solve the problem at all. The operator works. The question for a real search space is whether it supplies such blocks.

3 The real space and its fitness oracle

A NAS-Bench-101 cell is a directed acyclic graph on up to seven nodes: node 0 is the input, node $n-1$ the output, and the interior nodes are labeled `conv1x1`, `conv3x3`, or `maxpool3x3`. It has two orthogonal dimensions, its *wiring* (the adjacency) and its *labeling* (the ops), and a derived path view. The benchmark [12] tabulates the trained CIFAR-10 accuracy of all 423,624 cells, so fitness is a lookup and no network is trained during search.

The benchmark removes duplicate *isomorphic* graphs, so a recombined child is usually a relabeling of a catalogued cell rather than a verbatim match, and lookup needs a canonical form. We compute it by brute force: prune the nodes that lie on no input-to-output path, then take the lexicographically minimum adjacency-plus-labeling encoding over all $\leq 5! = 120$ permutations of the interior nodes. This is exact and needs no library. A self-test confirms it: over 5,000 cells both the verbatim graph and a random relabeling map to the same accuracy (5000/5000 each), and the 423,624 rows collapse to exactly 423,624 canonical keys, with no

collisions. The same canonicalization gives the alignment operator its node correspondence. The table is extracted from the official `nasbench_only108.tfrecord` by a small C decoder; best test 94.32% and best validation 95.06% reproduce the published optima.

4 Operators

Every individual is a seven-node upper-triangular cell. All arms share one population loop, one light mutation (flip an edge or re-roll an op), the same elitism, and the same query budget; only the crossover differs. Selection is on validation accuracy, and we report test accuracy. Four crossovers run against a random-search control:

- **flat**: per-cell uniform crossover. Each adjacency bit and each op comes from a uniformly chosen parent. It is structure-blind and maximally diverse. It also ignores that node k need not play the same role in both parents (the competing-conventions problem).
- **axis**: dimension-factored. The whole wiring comes from one parent, the whole labeling from the other.
- **aligned**: role-aligned. Match the second parent’s interior nodes to the first by role (op and in/out degree), then recombine node modules, each node’s incoming edges and its op taken together from one aligned parent. This removes the competing-conventions problem [10, 11].
- **path**: subgraph transplant. Recombine whole input-to-output paths, each carrying its own parent’s ops, added greedily under the nine-edge cap. This works along the graph dimension.

Matched compute, paired. Each arm gets a fixed query budget; the random control spends the same budget on independent random cells; a child that prunes to an invalid or over-budget graph is a spent query scored zero. In each trial the control and all arms share the same seed, so they start from an identical population and a trial is a paired observation. We report paired win/loss counts and the Wilcoxon signed-rank z of (arm minus baseline).

5 Results on the real space

Nothing meaningfully beats random. In absolute terms this is a space where random search is essentially unbeatable. Across every budget, random and all four crossovers finish within a few tenths of a percent of one another (Table 2), a gap comparable to the benchmark’s own training noise: the same architecture trained three times varies by a similar amount. No crossover produces a gain a practitioner would notice. What the paired, many-seed design buys is not a large effect but resolution: it lets us read a consistent ordering inside that noise band, which is what the rest of this section reports.

Structure does not help; at large budget it slips below random. Table 2 scales to population 50, budget 20,000, 4000 paired seeds. All differences are small (within the noise floor above) but consistent across the 4000 seeds. Flat is best at every budget and beats each structured operator head-to-head; its small edge over random shrinks as random saturates (z : 48, 46, 29, 13 across the four budgets) but stays consistent (2268/1322 paired wins even at 20,000). The structured operators worsen with compute: axis crosses from $z = +30$ to $z = -28$ (losing to random 1230/2559); path barely moves across a fortyfold budget rise (93.904 to 93.913) and ends at $z = -26$; aligned decays from $z = 42$ to $z = 1.9$, to break-even. So the structure-blind operator wins throughout, the two most structured (axis, path) fall below

Table 2: NAS-Bench-101, population 50, elite 12, 4000 paired seeds. Mean best test-accuracy (%) and Wilcoxon signed-rank z against random search (positive means above random). All gaps are small (within the benchmark’s training noise) but consistent across 4000 seeds. Flat is best at every budget; axis and path slip just below random at large budget; aligned’s edge decays to zero.

budget	random	flat (z)	axis (z)	aligned (z)	path (z)
500	93.645	93.998 (+48.0)	93.806 (+30.3)	93.917 (+41.8)	93.904 (+44.4)
2000	93.746	94.072 (+45.8)	93.872 (+22.5)	94.009 (+39.8)	93.911 (+29.9)
8000	93.941	94.085 (+29.2)	93.883 (−10.2)	94.036 (+19.7)	93.912 (−6.1)
20000	94.042	94.088 (+12.5)	93.886 (−27.6)	94.044 (+1.9)	93.913 (−26.0)

Table 3: Population sweep at fixed budget 2000, 600 paired seeds (elite = pop/4; the random control is identical throughout, mean 93.743). Mean best test-accuracy (%) and Wilcoxon z vs random: at pop 8 every arm loses to random; the gain rises with population.

pop	flat (z)	axis (z)	aligned (z)	path (z)
8	93.673 (−4.5)	93.652 (−5.7)	93.673 (−4.2)	93.526 (−12.6)
16	93.791 (+3.5)	93.715 (−1.8)	93.756 (+1.1)	93.706 (−2.7)
32	93.981 (+14.6)	93.823 (+5.9)	93.898 (+10.5)	93.840 (+7.3)
64	94.122 (+19.3)	93.917 (+11.6)	94.064 (+17.2)	93.957 (+14.2)

random at large budget, and the intermediate one ties it. This is not a validity artifact: axis, aligned, and path in fact yield more valid offspring (95 to 97 percent) than flat (90 to 96 percent).

Mechanism: productive diversity. Two independent measurements locate the cause. First, the final population’s genotypic spread (mean pairwise distance over 21 edges and 5 ops) shows two failure modes. The operators that end below random, axis and path, collapse the spread (1.8 and 1.3 against flat’s 2.1 at 20,000): they converge early, matching their plateaus. But aligned keeps the most spread of all (3.1, above flat) and still only ties random, because its variety never concentrates on good cells. Flat holds an intermediate, useful spread and wins. (We had expected the cleaner story that diversity ranks the operators; the measurement does not support it, and we report that.) Second, varying the population size at fixed budget sweeps the diversity available to crossover, and the comparison moves with it (Table 3): at population 8 every arm, flat included, loses to random ($z < 0$); the gain rises steadily until at population 64 all four beat it. Crossover here beats random exactly when it has enough diversity and an operator that turns diversity into good offspring rather than collapsing or wasting it.

Why random is hard to beat here: good cells are common. The small effect sizes have a simple structural cause, visible in the benchmark’s accuracy distribution. Good cells are common: of the 423,624 cells, 2.45% (10,378) score within one point of the best (94.32%), 17.7% within two points, and the median cell already reaches about 90%. So a single random draw is within one point of the best with probability 0.0245, and the best of just 100 random draws is within one point of the optimum 92% of the time. Random search reaches the good region almost at once, and at that resolution there is nothing left for a smarter search to win. The last sliver is the opposite case: only 0.09% (400 cells) lie within half a point of the best, so that final fraction of a percent is genuinely rare, and the differences there are inside the benchmark’s own training noise, so no method can resolve them reliably. Between the good region, reached at once by sampling, and the true optimum, rare and below the noise, there is no band where a directed search wins a real, repeatable margin. That is why random search is not beaten here, and why crossover’s edge is real as a direction but very small in size.

6 Relation to prior work

Nothing here is new in framing. Building-block recombination and crossover are Holland [2]; deceptive traps as their canonical test are Deb and Goldberg [3]; random search as a strong NAS baseline is [4, 9]; mutation-only evolutionary NAS is [7]; the competing-conventions problem the aligned operator targets motivates NEAT’s historical markings [10] and He et al.’s modular inheritable crossover [11]; NAS-Bench-101 [12] makes a study at this seed count possible. The contribution is a clean, reproducible pairing: a positive control that shows exactly when crossover’s building-block advantage appears, and a real NAS space where it does not. Four structure-respecting crossovers all lose (consistently, though by small margins) to plain uniform crossover, two of them to random search, with a diversity mechanism that explains why.

7 Discussion

One rule ties the two testbeds together: crossover helps to the extent the space is built from separable, recombinable blocks. The trap control makes the effect plain: crossover’s edge over mutation grows with the number of blocks until mutation cannot solve the problem. The real NAS cell does not supply such structure along its natural dimensions. Recombining its wiring and labeling, aligning nodes by role, and transplanting whole paths all lose to a structure-blind uniform crossover, whose own edge over random comes only from diversity, which the structural priors collapse or waste.

On effect size. None of the NAS-Bench-101 differences is large. Every method lands within a few tenths of a percent of random, inside the benchmark’s training noise, so no crossover meaningfully beats random search. Our claims are about the consistent direction of small effects resolved by thousands of paired seeds, not about their size. The practical message is the familiar one, that random search is very hard to beat on this space, made sharper by the fact that even the direction does not favor added structure.

Limitations. This is one tabulated cell space with a lookup fitness (no training-time confound, but also no guarantee of transfer to larger or hierarchical spaces). The aligned operator is one alignment, and a better one may exist; the burden is now on showing one that beats uniform crossover, which four natural attempts do not. The natural next step is a larger, hierarchical space where recombination has more room to matter.

Reproducibility

Every number regenerates from [13] with one command. The trap control is `bbtest.c` (`make bbtest`); the fitness table is extracted by `nb101_extract.c` (`make nb101_extract`); the search, oracle, and all operators are `validation/nb101.c` (`make nb101`), configurable through the environment (`POP`, `ELITE`, `SEEDS`, `BUDGETS`) and self-testing its oracle with `-selftest`. All build clean under `-std=c99 -pedantic` and run clean under AddressSanitizer and UBSan; a fixed xorshift generator makes every run reproducible.

References

- [1] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks*. Seminar in Artificial Intelligence, Open University of Israel, 1997 (revised digital edition, Zenodo 2026). <https://doi.org/10.5281/zenodo.20450526>
- [2] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [3] K. Deb, D. E. Goldberg. Analyzing Deception in Trap Functions. *Foundations of Genetic Algorithms* 2, 1993.

- [4] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [5] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [6] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [8] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [9] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.
- [10] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [11] C. He, H. Tan, S. Huang, R. Cheng. Efficient evolutionary neural architecture search by modular inheritable crossover. *Swarm and Evolutionary Computation* 64, 2021.
- [12] C. Ying, A. Klein, E. Real, E. Christiansen, K. Murphy, F. Hutter. NAS-Bench-101: Towards Reproducible Neural Architecture Search. *ICML* 2019.
- [13] SMBPANN reference implementation. <https://github.com/Anode1/SMBPANN>.